

Подготовка к защите Bridge

Сауле Джумабай Java 17 Assignment 3

Главная мысль: фигура хранит свои размеры и координаты, а отрисовщик определяет способ выполнения рисования. Мы можем заменить отрисовщик, сохранив тот же объект фигуры.

1 Что открыть и показать

Открой проект в IntelliJ IDEA с JDK 17 и запусти Main.main(). На Windows также можно открыть терминал в папке проекта и выполнить команды:

```
.\run.bat
.\test.bat
```

Сначала программа показывает круг с векторным отрисовщиком, затем тот же круг с растровым. После этого повторяет переключение для квадрата. Проверки должны закончиться строкой PASS: 9 test groups, 41 checks.

Файл	Что объяснить преподавателю
Renderer.java	Контракт: нарисовать окружность или линию.
Shape.java	Общие координаты и ссылка на Renderer. Здесь находится мост.
Circle.java	Радиус и один вызов renderCircle.
Square.java	Длина стороны и четыре вызова renderLine.
VectorRenderer.java	Выводит команды с дробными координатами.
RasterRenderer.java	Округляет координаты до пикселей.
Main.java	Создаёт объекты и меняет отрисовщик через setRenderer.

2 Самый важный участок

```
Shape circle = new Circle(5.4, 3.2, 2.6, vector);
circle.draw();
circle.setRenderer(raster);
circle.draw();
```

Первый draw вызывает реализацию VectorRenderer. setRenderer заменяет ссылку внутри фигуры. Второй draw вызывает RasterRenderer. Между этими двумя вызовами новый Circle не создаётся. Координаты и радиус остаются теми же.

Это симуляция рисования через консоль, как в лекции. Не называй её полноценным графическим движком: настоящие изображения программа не создаёт.

3 Речь на английском

Hello. This project demonstrates the Bridge design pattern in Java. The topic is shapes and renderers. There are two shapes, Circle and Square, and two rendering implementations, VectorRenderer and RasterRenderer.

The main idea is to separate what we draw from how we draw it. Shape is the abstraction. It stores common coordinates and a reference to the Renderer interface. Circle and Square extend Shape. The two renderer classes implement Renderer.

The reference inside Shape is the bridge. A shape uses composition to work with a renderer. It does not inherit from a renderer. The client chooses the concrete implementation when it creates the shape.

Now I will run the program. The circle is drawn with the vector renderer first. Then setRenderer replaces the renderer, and the same circle is drawn with the raster renderer. The same process is shown for the square. The vector renderer keeps decimal coordinates, while the raster renderer rounds them to pixel coordinates.

The square uses four line operations. This allows a new polygon, such as a triangle, to reuse the existing renderer interface. A new renderer can also implement the interface without changing the shape classes.

The code uses meaningful names, separate responsibilities, shared validation, dependency inversion and open/closed design. The behavior tests pass all forty-one checks. The main trade-off is extra classes and an additional level of delegation. Thank you.

Что означает эта речь

Ты представляешь две независимые части: фигуры и способы рисования. Показываешь ссылку Renderer внутри Shape и объясняешь композицию. Затем запускаешь программу и показываешь замену отрисовщика у того же объекта. Объясняешь четыре линии квадрата, расширение и пять принципов Clean Code. В конце называешь цену гибкости: дополнительные классы и вызов через другой объект.

Полезные слова

Термин	Простое значение
Abstraction	Высокоуровневая часть — Shape.
Implementation	Способ выполнения рисования.
Composition	Один объект хранит ссылку на другой и использует его.
Delegation	Передача выполнения операции другому объекту.
Runtime	Во время работы программы.
Decoupling	Уменьшение зависимости между частями.

4 Основные вопросы преподавателя

1 Где именно находится Bridge

В поле `private Renderer renderer` класса `Shape`. `Circle` и `Square` используют эту ссылку через `getRenderer()`. Сам мост — связь объектов через общий интерфейс.

The bridge is the Renderer reference stored in Shape.

2 Почему здесь композиция

Фигура содержит ссылку на отрисовщик и передаёт ему работу. `Circle` наследуется от `Shape`, но не от `VectorRenderer`. Наследование есть внутри одной иерархии, а две стороны связаны объектной ссылкой.

A shape has a renderer; it is not a renderer.

3 Зачем нужен интерфейс `Renderer`

Он задаёт общие операции. Фигуры могут вызывать их, не зная конкретного класса отрисовщика. Благодаря этому реализацию можно заменить.

Shapes depend on the interface, so implementations are interchangeable.

4 Как происходит переключение во время выполнения

Метод `setRenderer` получает другой объект `Renderer` и сохраняет его в поле. Следующий `draw` направляет операции уже новому объекту. Геометрия фигуры при этом не меняется.

setRenderer changes the target of the next drawing call.

5 Почему квадрат состоит из четырёх линий

Линия — низкоуровневая операция. Квадрат знает свои четыре ребра и передаёт их отрисовщику. Треугольник сможет использовать тот же `renderLine`, без нового метода `renderTriangle`.

A small set of primitives can support several different shapes.

6 Зачем конструктору `super`

`Circle` и `Square` вызывают конструктор `Shape`, чтобы установить общие `x`, `y` и `renderer`. Потом каждый класс проверяет и сохраняет свой размер: `radius` или `side`.

super initializes the common state inherited from Shape.

5 Вопросы посложнее

7 Чем Bridge отличается от Adapter и Strategy

Bridge разделяет две развивающиеся части системы. Adapter позволяет использовать существующий объект с несовместимым интерфейсом. Strategy делает взаимозаменяемыми алгоритмы поведения. Структура может быть похожей, поэтому объясняй именно цель.

8 Как добавить новый Renderer

Создать класс, который implements Renderer, реализовать renderCircle и renderLine, затем передать его фигуре. Менять Shape, Circle и Square не нужно. RecordingRenderer в тестах показывает такую замену.

9 Всегда ли новую фигуру можно добавить без изменений

Если её можно выразить существующими операциями — да. Треугольник состоит из трёх линий. Если нужна принципиально новая операция, интерфейс может потребовать изменения. Это ограничение контракта, его не стоит скрывать.

10 Какие проверки есть в программе

Проверяются вызовы отрисовщика, все четыре ребра квадрата, переключение и обратное переключение, сохранение геометрии, null, недопустимые размеры, вывод двух реализаций и независимое расширение. Всего 9 групп и 41 проверка.

11 Почему на UML белый ромб

Он обозначает общую ссылку, как в лекции. Main может передать один Renderer нескольким фигурам. Здесь объектная композиция означает has-a; жизненный цикл отрисовщика не принадлежит исключительно одной фигуре.

12 В чём недостатки и что можно улучшить

Дополнительные классы и делегирование делают маленькую программу сложнее. Растровая симуляция теряет дробную точность. Дальше можно добавить настоящий графический вывод и дополнительные фигуры, сохранив общий контракт там, где хватает его операций.

Перед защитой

Умей без подсказки назвать пять ролей Bridge и показать их в проекте. Проверь запуск на своём ноутбуке. Найди в коде каждый принцип Clean Code из отчёта. Подготовься изменить координаты или переключить отрисовщик обратно по просьбе преподавателя. Отвечай своими словами и называй ограничения программы прямо.